

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 4

ENSEMBLE LEARNING COMPARISONS

Reading Time	25 minutes
Prior Knowledge	Boosting methodology, Adaptive Boosting, Ensemble learning, Derivatives
Keywords	Pseudo residuals, loss function, learning rate, gradient boosting

The previous lesson introduced the reader to the details of boosting, specifically adaptive boosting. This lesson will explore Gradient Boosting and compare all ensemble learning models covered in this module to a common predictive problem.

1. Gradient Boosting

In the previous lesson we've looked at AdaBoost as it was among the first designed boosting algorithm with a particular loss function that can be sensitive to outliers. Gradient Boosting, however, is a generic algorithm that allows the optimization of an arbitrary loss function, thus making Gradient Boosting more flexible and robust to outliers than AdaBoost. The requirement of the loss function is that it is differentiable. We've seen that AdaBoost places more emphasis on predictions that were incorrect from the previous base learner although there is still some weight assigned to the correct predictions. With Gradient Boosting, however, there is only an emphasis on building the subsequent base learners based on the misclassifications from the previous learner. To illustrate this, we will go through an example to explain the process. Unlike the previous section, we will look at a regression example, i.e., a continuous target variable to illustrate boosting applied to a different type of problem. The dataset we use is the house price dataset from the UCI Machine Learning Repository. We will begin by importing the necessary libraries for this lesson along with the data.

```
In [1]: # import all necessary libraries

import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    RandomForestClassifier,
```

```

        StackingClassifier,
    )
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import mean_squared_error
    from sklearn.model_selection import train_test_split
    from sklearn.naive_bayes import GaussianNB

    # for stacking model later
    from sklearn.svm import SVC
    from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

    warnings.filterwarnings("ignore")

```

```

In [2]: data4gradboost = pd.read_excel("Real estate valuation data set.xlsx")

data4gradboost.set_index("No", drop=True, inplace=True)
data4gradboost.drop("X1 transaction date", axis=1, inplace=True)
data4gradboost.head()

```

```

Out[2]:

```

	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
No						
1	32.0	84.87882	10	24.98298	121.54024	37.9
2	19.5	306.59470	9	24.98034	121.53951	42.2
3	13.3	561.98450	5	24.98746	121.54391	47.3
4	13.3	561.98450	5	24.98746	121.54391	54.8
5	5.0	390.56840	5	24.97937	121.54245	43.1

Above shows a snapshot of the dataset, which has 5 predictors and a target variable in the last column that represents the price per unit area. Refer to <https://archive.ics.uci.edu/ml/datasets/Real+estate+valuation+data+set> for more details on this dataset. We start by separating the dataset into the features and target column for a train/test split. The size of the test set is not important for this example since we are only interested in understanding the gradient boosting algorithm methodology.

```

In [3]: # Separate into X and Y
X = data4gradboost.iloc[:, :-1]
y = data4gradboost.iloc[:, -1]

```

```

In [4]: X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=2)

```

1.1 Step 1: Fit Weak Learner

Now that we have the data to develop our gradient boosting model, we can begin the first step. Similarly, for the AdaBoost in the previous lesson, we use weak decision tree classifiers as the base learners. We choose a max_depth of 2 and then fit the base learner to the training dataset.

```
In [5]: tree_1 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_1.fit(X_train, y_train)
```

```
Out[5]: ▾ DecisionTreeRegressor
DecisionTreeRegressor(max_depth=2, random_state=2)
```

1.2 Step 2: Calculate Residuals

Once we fit our first weak learner to the data, we then calculate the residuals. The residuals are simply given by

$$\text{residuals} = y_{\{i\}} - \hat{y}_{\{i\}}$$

where $y_{\{i\}}$ is the actual outcome and $\hat{y}_{\{i\}}$ is the predicted value for observation i . The residuals now become the new target data to train the next base learner. Gradient boost builds trees based on the residuals or errors of the previous tree.

1.3 Step 3: Train Next Base Learner on Residuals

```
In [6]: # predictions
y_pred1 = tree_1.predict(X_train)
# residuals become the next target data to train
y2_train = y_train - y_pred1
```

Once we have the new target values to train the next base learner, we continue this cycle, i.e., steps 1 to 3. In this example, we will do this for 3 base learners, i.e., 3 iterations.

```
In [7]: # initialize new tree or 2nd base Learner
tree_2 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_2.fit(X_train, y2_train)

# predictions
y_pred2 = tree_2.predict(X_train)
# new target values
y3_train = y2_train - y_pred2

# initialize new tree or 3rd base Learner
tree_3 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_3.fit(X_train, y3_train)

# last set of predictions. Stop at 3rd base Learner
y_pred3 = tree_3.predict(X_train)
```

1.4 Final Step: Calculate Final Predictions

Finally, for predictions of unseen observations, we would add the predictions of each base learner as below. We assign the final predictions to `y_pred` and then calculate a performance metric, namely Root Mean Squared Error (RMSE). We obtain an RMSE of 9.9625.

```
In [8]: # Make predictions on test set for all 3 base Learners
y1_pred = tree_1.predict(X_test)
```

```

y2_pred = tree_2.predict(X_test)
y3_pred = tree_3.predict(X_test)

y_pred = y1_pred + y2_pred + y3_pred

# check MSE
round(mean_squared_error(y_test, y_pred) ** 0.5, 4)

```

Out[8]: 9.9625

Building the gradient boosting algorithm from scratch is quite tedious, and it is easy to make an error coding it. Fortunately, Python has the built-in `GradientBoostingRegressor` from sklearn to make things much easier. Below is the more elegant and efficient way to code this. Notice the hyperparameter `learning_rate` = 1, which we'll address later.

```
In [9]: from sklearn.ensemble import GradientBoostingRegressor
```

```
In [10]: gbr = GradientBoostingRegressor(
          max_depth=2, n_estimators=3, random_state=10, learning_rate=1
        )
```

```
In [11]: gbr.fit(X_train, y_train)
```

```
Out[11]: ▼ GradientBoostingRegressor
          GradientBoostingRegressor(learning_rate=1, max_depth=2, n_estimators=3,
                                     random_state=10)
```

The RMSE results are identical to our model built from scratch.

```
In [12]: gbr_pred = gbr.predict(X_test)
          round(mean_squared_error(y_test, gbr_pred) ** 0.5, 4)
```

Out[12]: 9.9625

The example above illustrates the procedure followed by a gradient boosting algorithm. Below will show how it works with a bit more attention to the mathematics behind it.

1.5 Gradient Boosting Reasoning

The algorithm begins with a naive guess of using the average, say $\langle y \rangle$, for all house prices or target values. The reasoning behind this is gradient boosting aims to minimize the loss function $L(y_i, \hat{y}_i)$. A common L used is of the form

$$L(y_i, \hat{y}_i) = \frac{1}{2} \sum_{i=1}^N \left(y_i - \hat{y}_i \right)^2 \tag{1.1}$$

over N observations. This type of loss function is an L2 Loss function.

At the start, we assume $\hat{y}_i$ is a constant function $F_0(x)$ over predictors x . From calculus, minimizing implies finding the derivative of (1.1) with respect to $F_0(x)$,

equating to zero and solving for $F_0(x)$. If we do this, the solution $F_0^*(x)$ is,
$$F_0^*(x) = \frac{1}{N} \sum_{i=1}^N y_i = \bar{y} \tag{1.2}$$

which is how we get the average as the first guess. Note, had we chosen a different L this may not have been the case. For illustrative purposes, we will use the loss function in (1.1) to explain the concept.

Remember from our example in section 1.1 that the next base learner would aim to predict the residuals of the previous base learner instead of the actual values. Starting off with predictions as the average of the actuals is a very simplistic estimation; however, the next iteration 1 will improve on the previous base learner predictions such that new predictions are
$$F_1(x) = F_0(x) + h_1(x) \tag{1.3}$$

where $h_1(x)$ is an added estimator such that together with the previous estimate F_0 , will improve the estimates. We hope that our new estimates would predict the actuals i.e.,
$$F_1(x) = F_0(x) + h_1(x) = y \tag{1.4}$$

but rearranging (1.4) gives us $h_1(x) = y - F_0(x)$. This is just the residuals from the previous base learner's predictions and is what we were doing in section 1.1. So to get this estimator $h_1(x)$ we would provide as a target column $(y - F_0(x))$ and fit a base learner on the dataset $\{(x_i, y_i - F_0(x))\}_{i=1}^N$ or $\{(x_i, r_{i1})\}_{i=1}^N$ where r_{i1} are the pseudo residuals at iteration 1. In general at iteration j , we have from (1.4),

$$F_j(x) = F_{j-1}(x) + h_j(x) \tag{1.5}$$

and we develop our estimator $h_j(x)$ in the same way. There is a parameter ν , that is not shown in (1.5) where,

$$F_j(x) = F_{j-1}(x) + \nu h_j(x) \tag{1.6}$$

which we've set as $\nu = 1$ but can actually take on values in the range $[0,1]$. This parameter is the *learning rate*, which improves accuracy in the long run. Think of it as a step size, and often many learners with small step sizes provide better results. From recursive substitution over all iterations, the last iteration T will provide a final model such that for a test or unseen data point x^* the prediction would be

$$F_T(x^*) = F_0 + \sum_{j=1}^{T-1} \nu h_j(x^*), \tag{1.6}$$

which ties in to the last step for our gradient boosting model built from scratch.

Classification is very similar to the regression process but differs mainly in the scoring. The reader is referred to Murphy (607) for further reading on classification. There is also a modified gradient boosting algorithm, which improves on what we've covered above called Extreme Gradient Boosting or XGBoost. One of the modifications is that XGBoost uses a 2nd order approximation of the loss function and adds a regularizer on the tree complexity. Refer to Murphy (615) for details on this.

2. Ensemble Learning Comparison

This section will apply Bagging, Stacking, AdaBoost, and Gradient boosting to a common classification problem. We predict whether the Luxembourg index (LUXXX) will exceed a return of 0.25% in any direction. The predictors are a combination of country indices and technical indicators. We start by importing the data.

```
In [13]: import warnings

warnings.filterwarnings("ignore")

# Loc = "ENTER YOUR FULL PATH TO LOCATION OF DATA FILE HERE"
# data_df = pd.read_csv(Loc+"/MScFE 650 MLF GWP Data.csv")

data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
# Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

Create the target variable.

```
In [14]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date", target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()

# Create Target Column.
# Shift period for target column
data_df[target ETF + "_returns" + "_Shift"] = data_df[target ETF + "_returns"].s

# Strategy to take Long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)
```

Technical indicators included as predictors are slow to fast moving average ratio (SMA_ratio), Relative Strength Index (RSI), and Rate of Change (RC).

```
In [15]: # Four country indices used.
feats = ["MSCI KOREA", "MSCI DENMARK", "MSCI FRANCE", "MSCI NORWAY"]

# creating the technical indicators
data_df["SMA_5"] = data_df[target ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)

# shift the price of the target by 1 unit previous in time
```

```

data_df["Diff"] = data_df[target ETF] - data_df[target ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up", "avg_15down"],
    axis=1,
    inplace=True,
)

data_df["RC"] = data_df[target ETF].pct_change(15)

# all_feats
feats.append("SMA_ratio")
feats.append("RSI_ratio")
feats.append("RC")

```

Now we can apply all our models to the data. Perform the train/test split of 80/20.

```

In [16]: # Train/Test split
# Train/Test split. No NaNs in the data.
NoNaN_df = data_df.dropna()
X = NoNaN_df[feats]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

```

We will tune the boosting models and use the hyperparameters from Lesson 2 for the bagging model. We should note that AdaBoost by default has Decision Tree Classifiers as base learners with `max_depth = 1` as default. For the gradient boosting classifier, we assume the default `max_depth` of 3. We only tune on learning rate and `n_estimators` to reduce computational time. We also do the same for the XGBoost model.

```
In [17]: from sklearn.model_selection import GridSearchCV
```

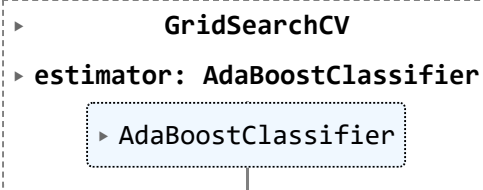
```
In [18]: # parameters for AdaBoost
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1, 0.25, 0.

gridAdBoost = GridSearchCV(
    AdaBoostClassifier(), param_grid, refit=True, verbose=3, cv=3
)
# fitting the model for grid search
gridAdBoost.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

```
[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.710 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.603 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.694 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.667 total time= 0.1s
[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.571 total time= 0.1s
[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.661 total time= 0.1s
[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.694 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.645 total time= 0.1s
[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.651 total time= 0.1
s
[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.540 total time= 0.1
s
[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.677 total time= 0.1
s
[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.603 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=50;; score=0.587 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=50;; score=0.629 total time= 0.1s
[CV 1/3] END learning_rate=0.5, n_estimators=100;; score=0.619 total time= 0.1s
[CV 2/3] END learning_rate=0.5, n_estimators=100;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=0.5, n_estimators=100;; score=0.613 total time= 0.1s
[CV 1/3] END learning_rate=1.0, n_estimators=10;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=10;; score=0.603 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=10;; score=0.597 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=20;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=20;; score=0.651 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=20;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=50;; score=0.587 total time= 0.1s
[CV 2/3] END learning_rate=1.0, n_estimators=50;; score=0.540 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=50;; score=0.629 total time= 0.1s
[CV 1/3] END learning_rate=1.0, n_estimators=100;; score=0.635 total time= 0.1s
[CV 2/3] END learning_rate=1.0, n_estimators=100;; score=0.587 total time= 0.1s
[CV 3/3] END learning_rate=1.0, n_estimators=100;; score=0.661 total time= 0.1s
```

Out[18]:



```
In [19]: # print best parameter after tuning
print(gridAdBoost.best_params_)

# print how our AdaBoost model looks after hyper-parameter tuning
print(gridAdBoost.best_estimator_)

{'learning_rate': 0.1, 'n_estimators': 10}
AdaBoostClassifier(learning_rate=0.1, n_estimators=10)

In [20]: # parameters for Gradient Boosting Classifier
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1, 0.25, 0.

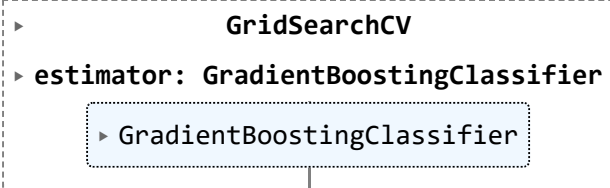
GBgrid = GridSearchCV(
    GradientBoostingClassifier(), param_grid, refit=True, verbose=3, cv=3
)

# fitting the model for grid search
GBgrid.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

```
[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.619 total time= 0.1s
[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.635 total time= 0.1s
[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.694 total time= 0.1s
[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.556 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.651 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.694 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.603 total time= 0.1
s
[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.587 total time= 0.1
s
[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.645 total time= 0.1
s
[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.587 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=50;; score=0.571 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=50;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=50;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=100;; score=0.603 total time= 0.1s
[CV 2/3] END learning_rate=0.5, n_estimators=100;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=0.5, n_estimators=100;; score=0.645 total time= 0.1s
[CV 1/3] END learning_rate=1.0, n_estimators=10;; score=0.556 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=10;; score=0.556 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=10;; score=0.613 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=20;; score=0.571 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=20;; score=0.476 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=20;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=50;; score=0.587 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=50;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=100;; score=0.571 total time= 0.1s
[CV 2/3] END learning_rate=1.0, n_estimators=100;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=1.0, n_estimators=100;; score=0.710 total time= 0.1s
```

Out[20]:



```
In [21]: # print best parameter after tuning
print(GBgrid.best_params_)

# print how our Gradient Boost model looks after hyper-parameter tuning
print(GBgrid.best_estimator_)

{'learning_rate': 0.1, 'n_estimators': 50}
GradientBoostingClassifier(n_estimators=50)
```

```
In [22]: # import library for XGBoost
import xgboost as xgb
```

```
In [23]: # parameters for XG Boosting Classifier
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1, 0.25, 0.

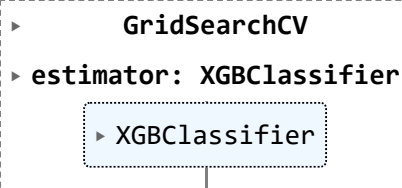
XGB_model = xgb.XGBClassifier()
XGBgrid = GridSearchCV(XGB_model, param_grid, refit=True, verbose=3, cv=3)

# fitting the model for grid search
XGBgrid.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

```
[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.698 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.651 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.667 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.651 total time= 0.0
s
[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.635 total time= 0.0
s
[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.629 total time= 0.0
s
[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.651 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.651 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=50;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=50;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=100;; score=0.651 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=100;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=100;; score=0.613 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=10;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=10;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=10;; score=0.597 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=20;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=20;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=20;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=50;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=50;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=50;; score=0.613 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=100;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=100;; score=0.556 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=100;; score=0.645 total time= 0.0s
```

Out[23]:



```
In [24]: # print best parameter after tuning
print(XGBgrid.best_params_)

# print how our XGBoost model looks after hyper-parameter tuning
print(XGBgrid.best_estimator_)

{'learning_rate': 0.25, 'n_estimators': 10}
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.25, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=10,
              n_jobs=None, num_parallel_tree=None, ...)
```

So we find that AdaBoost and Gradient Boosting have the same optimal hyperparameters; however, XGBoost has a different learning rate. We can train all of our ensemble models and then compare performance afterwards.

```
In [25]: # Train with Tuned Random Forest
# Create a tuned RF Classifier
bagmodel_tuned = RandomForestClassifier(
    max_depth=2, min_samples_split=8, n_estimators=10, random_state=10
)

bagmodel_tuned.fit(X_train, y_train)
```

```
Out[25]: ▼ RandomForestClassifier

RandomForestClassifier(max_depth=2, min_samples_split=8, n_estimators=10,
                       random_state=10)
```

This is the stacking model.

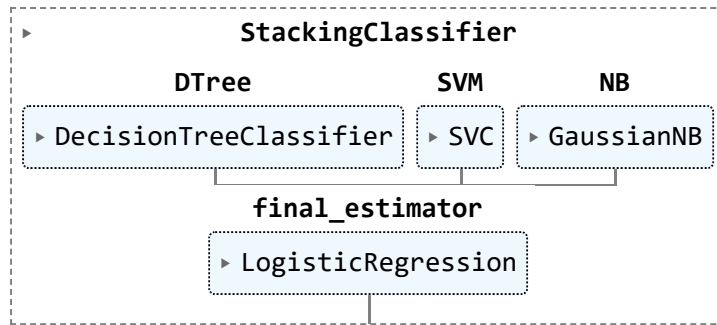
```
In [26]: clf1 = DecisionTreeClassifier() # Decision Tree

clf2 = SVC(kernel="rbf") # Support Vector Classifier

clf3 = GaussianNB() # Gaussian Naive Bayes

est_rs = [("DTree", clf1), ("SVM", clf2), ("NB", clf3)]
# Meta model
mylr = LogisticRegression()
# creating a stacking classifier
stackingCLF = StackingClassifier(
    estimators=est_rs, final_estimator=mylr, stack_method="auto", cv=3
)
stackingCLF.fit(X_train, y_train)
```

Out[26]:



Fit all tuned boosting models to the training set.

```

In [27]: # Create a tuned AdaBoost Classifier
AdaBoost_tuned = AdaBoostClassifier(learning_rate=0.1, n_estimators=10)

# Create a tuned Gradient Boosting Classifier
GB_tuned = GradientBoostingClassifier(learning_rate=0.1, n_estimators=10)

# Create a tuned XGBoost Classifier
XGB_tuned = xgb.XGBClassifier(learning_rate=0.25, n_estimators=10)

# train boosting models
AdaBoost_tuned.fit(X_train, y_train)
GB_tuned.fit(X_train, y_train)
XGB_tuned.fit(X_train, y_train)
print("Training complete")

```

Training complete

Lastly, we compare all models with an ROC curve.

```

In [28]: from sklearn.metrics import roc_auc_score, roc_curve

```

```

In [29]: # predicted probabilities generated by models
y_pred_probaStack = stackingCLF.predict_proba(X_test) # stacking
y_pred_probaRF = bagmodel_tuned.predict_proba(X_test) # RF
y_pred_probaAdB = AdaBoost_tuned.predict_proba(X_test) # AdaBoost
y_pred_probaGb = GB_tuned.predict_proba(X_test) # `GradBoost`
y_pred_probaXGB = XGB_tuned.predict_proba(X_test) # XGBoost

# Stacking ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_probaStack[:, 1])
auc = round(roc_auc_score(y_test, y_pred_probaStack[:, 1]), 4)

# RF ROC dependencies
fpr_RF, tpr_RF, _ = roc_curve(y_test, y_pred_probaRF[:, 1])
auc_RF = round(roc_auc_score(y_test, y_pred_probaRF[:, 1]), 4)

# AdaBoost ROC dependencies
fpr_AB, tpr_AB, _ = roc_curve(y_test, y_pred_probaAdB[:, 1])
auc_AB = round(roc_auc_score(y_test, y_pred_probaAdB[:, 1]), 4)

# Gradient Boost ROC dependencies
fpr_GB, tpr_GB, _ = roc_curve(y_test, y_pred_probaGb[:, 1])
auc_GB = round(roc_auc_score(y_test, y_pred_probaGb[:, 1]), 4)

# XGB ROC dependencies
fpr_XGB, tpr_XGB, _ = roc_curve(y_test, y_pred_probaXGB[:, 1])

```

```

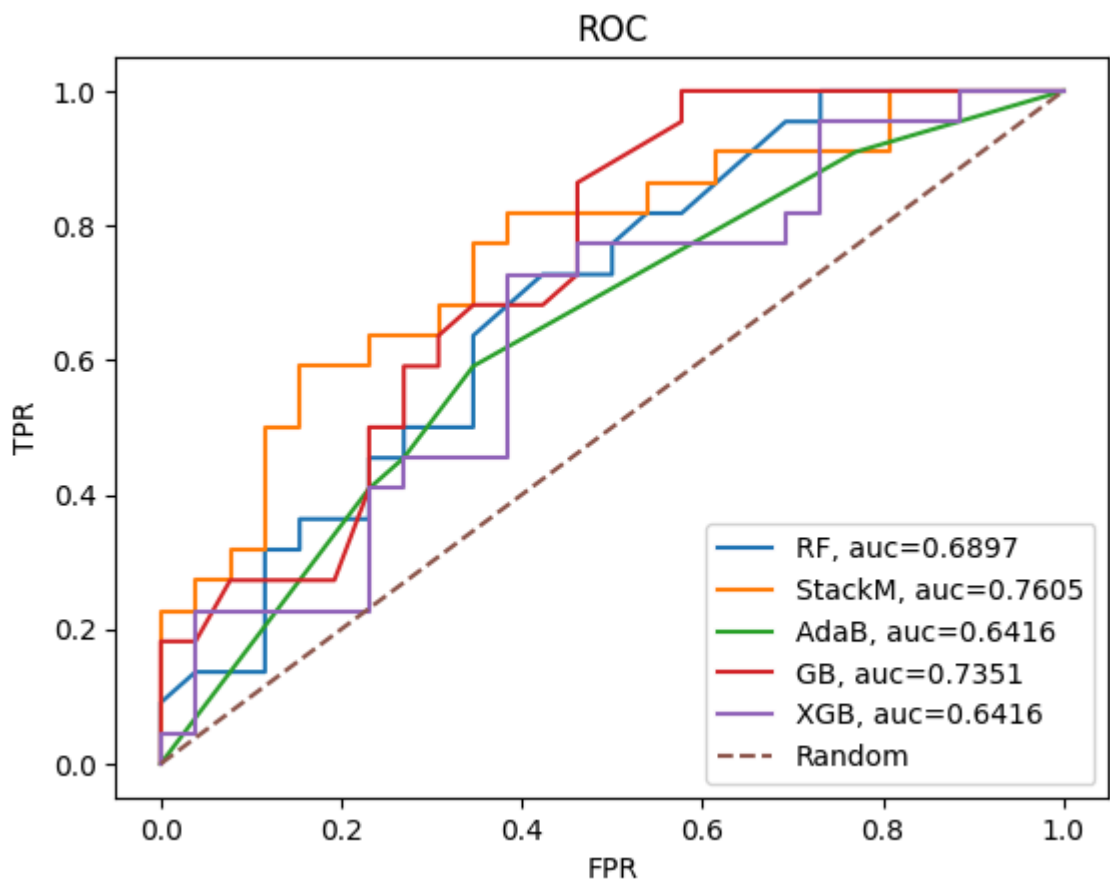
auc_XGB = round(roc_auc_score(y_test, y_pred_probaXGB[:, 1]), 4)

# RF Model
plt.plot(fpr_RF, tpr_RF, label="RF, auc=" + str(auc_RF))
# Stacking Model
plt.plot(fpr, tpr, label="StackM, auc=" + str(auc))
# AdaBoost Model
plt.plot(fpr_AB, tpr_AB, label="AdaB, auc=" + str(auc_AB))
# `GradBoost` Model
plt.plot(fpr_GB, tpr_GB, label="GB, auc=" + str(auc_GB))
# XGBoost Model
plt.plot(fpr_XGB, tpr_XGB, label="XGB, auc=" + str(auc_XGB))

# Random guess model
plt.plot(fpr, fpr, "--", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()

```



Great! All models performed reasonably well with some models exceeding 70% AUC. Keep in mind that we did not explore all hyperparameters for models, but this is a good enough starting point to create your own models and explore the hyperparameter space and effect on model performance.

3. Conclusion

This lesson explored the last ensemble learning algorithm, namely gradient boosting in more detail after briefly mentioning it in Module 3. We compared all ensemble learning algorithms covered in this module in a common classification problem and obtained results significantly better than a no-skill model, thus showing the added value in using the combined predictive power of weak learners.

References

1. Murphy, Kevin P. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
2. University of California, Irvine Machine Learning Repository. "Real Estate Valuation Data Set." <https://archive.ics.uci.edu/ml/datasets/Real+estate+valuation+data+set>

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.